



[논문 리뷰] Attention is all you need

rachelses · 2026년 3월 16일

통계 수정 삭제



NLP

▼ 목록 보기

1/1

DOI : <https://arxiv.org/abs/1706.03762>

Summary

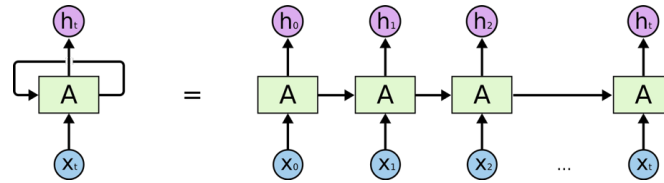
최근 자연어 처리 분야에서 시퀀스 변환 문제는 주로 순환 신경망(RNN) 기반의 인코더-디코더 구조에 의해 해결되어 왔으나, 이러한 모델은 순차적 계산 방식으로 인해 병렬화가 어렵고 긴 시퀀스를 처리하는 데 한계가 있다. 본 논문에서는 이러한 한계를 극복하기 위해, 순환이나 합성곱 연산에 의존하지 않고 전적으로 어텐션 메커니즘만을 기반으로 하는 새로운 아키텍처인 Transformer를 제안한다. Transformer는 멀티헤드 셀프 어텐션을 통해 입력 시퀀스 내 장거리 의존성을 효과적으로 모델링하면서도 높은 병렬성을 달성하며, 기계 번역 실험에서 기존의 RNN 및 CNN 기반 모델을 뛰어넘는 성능을 보여 기존 시퀀스 모델링 방식에 대한 근본적인 대안을 제시한다.

Background

기존 모델들의 한계

RNN/LSTM 기반 Seq2Seq 모델

기본 구조



RNN/LSTM 기반 Seq2Seq는 Encoder-Decoder 구조를 사용한다.

Encoder는 입력 문장을 앞에서부터 순서대로 읽으면서 hidden state에 정보를 압축하고, Decoder는 이 정보를 바탕으로 출력 문장을 한 단어씩 생성한다. 즉, 입력과 출력을 모두 순차적으로 생성하는 구조다.

예를 들어 번역 작업에서는,

- Encoder가 입력 문장을 차례대로 읽고
- 각 시점의 정보를 hidden state에 반영한 뒤
- 마지막 표현 또는 encoder의 출력들을 decoder에 전달하고
- Decoder가 이전에 생성한 단어와 encoder 정보를 참고해 다음 단어를 예측한다.

이런 흐름으로 동작한다.

LSTM은 기본 RNN보다 긴 문맥 정보를 더 잘 유지하도록 만든 변형이지만, 여전히 순서대로 처리해야 하는 구조 자체는 같다.

한계

1. 순차 처리이기 때문에 병렬화 어려움

RNN/LSTM은 현재 시점의 계산이 이전 hidden state에 의존한다.

그래서 문장 전체를 한 번에 처리하지 못하고, 토큰을 하나씩 순서대로 계산해야 한다. 논문은 이런 순차적 특성이 training example 내부 병렬화를 막는다고 지적한다.

2. Long-term dependency 문제

앞쪽 단어 정보가 뒤쪽 단어에 도달하려면 여러 시점을 거쳐야 한다.

이처럼 정보 전달 경로가 길어질수록, 멀리 떨어진 단어 관계를 학습하기가 어려워진다. 논문은 self-attention이 이런 long-range dependency를 더 짧은 경로로 연결할 수 있다는 점을 강조한다.

3. 순차 구조의 계산적 한계

문장을 순차적으로 처리해야 하므로, GPU를 활용한 대규모 병렬 연산에 불리하다.

문장이 길어질수록 이 단점은 더 커지고, 학습 시간과 차원 측면에서도 비효율이 커진다. Transformer는 바로 이 문제를 줄이기 위해 recurrence 없이 attention 중심 구조를 제안했다.

RNN with Attention

기본 구조

RNN/LSTM 기반 Seq2Seq에 attention 메커니즘을 추가한 구조다.

Encoder는 입력 문장을 순서대로 읽어 각 시점의 hidden state를 만들고, Decoder는 출력 문장을 한 단어씩 생성한다. 이때 Decoder는 마지막 hidden state 하나에만 의존하지 않고, 매 시점마다 encoder의 여러 hidden state를 다시 참고해 필요한 정보를 가져온다. 논문은 attention 메커니즘이 다양한 sequence transduction 모델에서 중요한 요소가 되었지만, 대부분은 recurrent network와 함께 사용되었다고 설명한다.

이 구조에서는 decoder가 다음 단어를 생성할 때,

- 현재 decoder 상태를 기준으로
- encoder의 각 hidden state와의 관련도를 계산하고
- 그 중 중요한 위치에 더 큰 가중치를 주어
- 가중합된 문맥 정보(context)를 만든 뒤
- 이를 이용해 다음 단어를 예측한다.

즉, 입력 문장을 하나의 벡터로만 압축하지 않고, 필요한 시점마다 입력의 여러 부분을 선택적으로 참고할 수 있게 한 구조이다.

이 모델의 장점은 입력 문장을 하나의 고정 길이 벡터에만 압축하지 않고, decoder가 매 시점마다 encoder의 여러 hidden state를 다시 참고할 수 있다는 것이다. 이를 통해 긴 문장에서도 중요한 정보를 더 유연하게 활용할 수 있고, 기존 Seq2Seq의 정보 압축 한계를 어느 정도 완화할 수 있다.

한계

1. 여전히 순차 구조

Attention이 추가되었더라도, encoder와 decoder의 본체는 여전히 RNN/LSTM이다. 따라서 토큰을 순서대로 처리해야하는 특성은 그대로 남아있다.

2. 병렬화가 어렵고 학습 속도 느림

Attention은 정보 접근을 더 유연하게 만들었지만, 연산 구조 자체를 병렬적으로 바꾸지는 못했다. 즉, decoder는 여전히 이전 상태에 의존하며 한 단계씩 출력해야 하고, encoder 역시 순차적으로 입력을 읽는다. 그래서 GPU를 활용한 대규모 병렬 연산에는 한계가 있다.

3. Long-term dependency 문제를 완전히 해결하지는 못함

Attention 덕분에 필요한 입력 위치를 직접 참고할 수는 있지만, 모델의 기본 뼈대가 recurrent 구조이기 때문에 정보가 시간축을 따라 전달되는 특성 자체는 남아 있다.

논문은 Transformer가 self-attention만으로 장거리 의존성을 더 짧은 경로로 연결할 수 있다고 강조한다.

CNN 기반 Seq2Seq 모델

기본 구조

RNN 대신 convolution layer를 사용해 입력과 출력을 처리하는 sequence-to-sequence 모델이다. 입력 문장은 convolution 연산을 통해 각 위치의 표현을 계산하고, 여러 층을 쌓으면서 더 넓은 문맥 정보를 반영한다. Decoder 역시 convolution 기반으로 다음 단어를 생성한다.

이 구조에서는 각 단어가 주변 단어들과의 지역적 관계(local pattern)를 먼저 반영하고, 층이 깊어질수록 더 넓은 범위의 문맥을 보게 된다.

즉, RNN처럼 한 시점씩 순서대로 hidden state를 넘기는 대신, 합성곱 연산으로 여러 위치를 병렬적으로 처리하면서 문장 표현을 만든다.

이 모델의 장점은 RNN보다 병렬화에 유리하다는 것이다. 합성곱 연산은 각 위치에 대해 동시에 계산할 수 있기 때문에, 순차 처리에 묶이는 RNN 계열보다 GPU를 활용한 학습이 더 효율적이다. 그래서 계산 속도 측면에서는 RNN 기반 모델의 한계를 일부 줄일 수 있다.

한계

1. 먼 단어 관계를 직접 연결하기 어려움

CNN은 기본적으로 가까운 위치의 정보부터 합성곱으로 묶어 나간다. 따라서 멀리 떨어진 두 단어의 관계를 반영하려면 여러 convolution layer를 거쳐야 한다.

논문은 두 위치를 연결하는 데 필요한 연산 수가 거리와 함께 증가한다고 설명하며, Transformer의 self-attention은 이를 상수 경로로 줄인다고 비교한다.

2. 긴 문맥을 볼수록 구조가 깊어질 수 있음

한 번의 convolution은 제한된 범위만 보기 때문에, 전체 문장을 충분히 반영하려면 더 많은 층이나 더 넓은 receptive field가 필요하다. 즉, 긴 거리 의존성을 학습하려면 구조가 점점 복잡해질 수 있다.

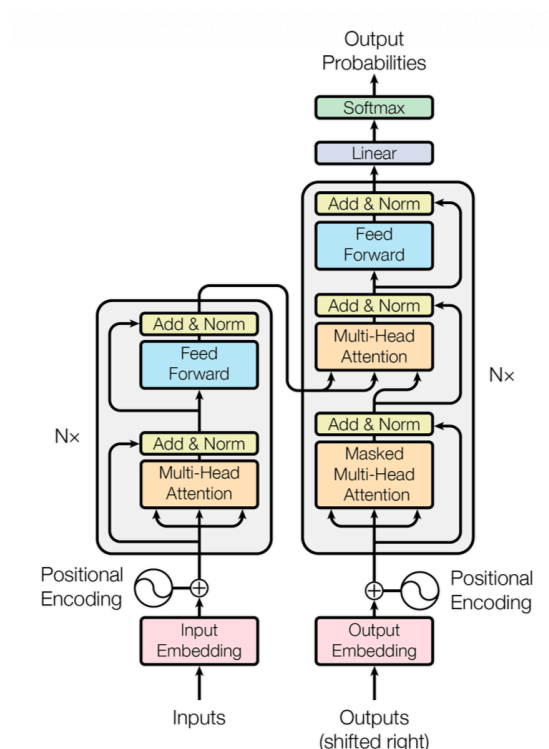
3. 전역 관계를 한 번에 보기 어렵다

CNN은 지역 정보를 점진적으로 확장하는 방식이라, 문장 전체의 모든 단어 관계를 한 번에 직접 계산하지는 못한다.

반면 Transformer의 self-attention은 각 단어가 다른 모든 단어를 직접 참고할 수 있어 전역 관계를 더 직접적으로 반영할 수 있다.

Model Architecture

Transformer 전체 구조



Embeddings

Transformer는 텍스트를 직접 처리할 수 없기 때문에, 입력 토큰을 먼저 embedding을 통해 벡터로 변환한다.

각 단어는 고정된 차원 d_{model} 의 벡터로 표현되며, 이 벡터가 모델의 입력으로 사용된다. 이렇게 변환된 벡터는 이후 positional encoding과 함께 사용되어 encoder의 입력으로 들어간다.
또한 Transformer에서는 입력 embedding과 출력 embedding, 그리고 softmax 이전의 선형 변환이 같은 weight를 공유한다. 이를 통해 모델의 파라미터 수를 줄이고, 표현을 더 일관되게 학습할 수 있다.

Encoder-Decoder

Encoder

입력 문장을 받아 각 단어가 문맥을 반영한 표현으로 변환하는 역할을 한다. 여러 개의 encoder layer가 쌓여 있으며, 각 layer는 self-attention과 feed-forward network로 구성된다. self-attention을 통해 각 단어는 문장 안의 다른 모든 단어를 참고할 수 있고, 이를 통해 입력 전체의 관계를 반영한 표현을 만든다.
논문에서는 encoder를 동일한 6개 layer의 stack으로 구성하고, 각 layer가 multi-head self attention과 position-wise feed-forward network를 포함한다고 설명한다.

Decoder

Encoder가 만든 표현을 참고하여 출력 문장을 한 단어씩 생성하는 역할을 한다. Decoder 역시 여러 개의 layer로 이루어지며, masked self-attention, encoder-decoder attention, feed-forward network를 포함한다. 여기서 masked self-attention은 미래 단어를 미리 보지 못하게 가리고, encoder-decoder attention은 입력 문장 정보 중 현재 출력에 필요한 부분을 참고하게 한다.
논문은 decoder를 동일한 6개 layer의 stack으로 구성하며, encoder 출력에 대한 attention sub-layer를 추가한다고 설명한다.

Position-wise Feed-Forward Network

Transformer의 각 encoder와 decoder layer에는 attention 이후에 Position-wise Feed-Forward Network(FFN)가 추가로 적용된다. 이 네트워크는 각 위치(token)에 대해 독립적으로 동일하게 적용되는 완전연결 신경망이다. 즉, attention이 단어들 간의 관계를 반영한다면, FFN은 각 단어의 표현을 한 번 더 변환하고 정제하는 역할을 한다.
구조는 간단하게 두 개의 선형 변환과 그 사이의 비선형 활성화 함수(ReLU)로 이루어져 있다.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

이 연산은 모든 위치에 대해 동일한 방식으로 적용되지만, 각 layer마다 다른 파라미터를 사용한다. 따라서 같은 구조를 반복하면서도 점점 더 풍부한 표현을 학습할 수 있다.

또한 FFN은 각 위치를 독립적으로 처리하기 때문에, sequence 전체를 동시에 병렬적으로 계산할 수 있다. 이러한 구조는 convolution에서 kernel size가 1인 연산과 유사하게 볼 수 있으며, 각 단어의 표현을 개별적으로 변환하는 역할을 한다.

Softmax

Decoder의 출력은 바로 단어가 되는 것이 아니라, 먼저 선형 변환(linear transformation)을 거쳐 각 단어에 대한 점수(logit)로 변환된다. 이후 softmax 함수를 적용해 전체 vocabulary에 대한 확률 분포로 바꾼다. 이 확률 분포를 바탕으로 모델은 다음에 올 단어를 예측하며, 가장 높은 확률을 가진 단어를 선택하거나 샘플링을 통해 문장을 생성한다.

또한 Transformer에서는 이 선형 변환이 embedding과 weight를 공유하도록 설계되어, 입력과 출력 표현을 일관되게 유지한다.

Attention

Attention 이란?

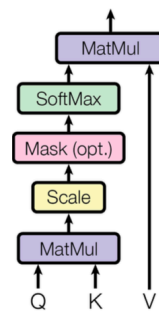
기존 Encoder-Decoder 구조에서는 입력 문장을 하나의 고정된 벡터로 압축한 뒤 Decoder가 이를 기반으로 출력을 생성했다. 하지만 문장이 길어질수록 이 고정 벡터에 모든 정보를 담는 것이 어려웠고, 중요한 정보가 손실되는 문제가 발생했다.

Attention은 이러한 정보 병목(bottleneck) 문제를 해결하기 위해 등장했다.

문장을 처리할 때, 전체 입력을 동일하게 사용하는 것이 아니라 **지금 더 중요하게 참고해야 할 부분에 집중하는 방식**이다. 즉, 현재 단어가 문장 안의 다른 단어들 중 무엇을 얼마나 참고할지를 결정하는 매커니즘이다.

Scaled Dot-Product Attention

Scaled Dot-Product Attention



Transformer에서 사용하는 기본 attention 연산은 Scaled Dot-Product Attention이다. 입력은 Query(Q), Key(K), Value(V)로 이루어진다. 먼저 Query와 모든 Key의 내적(dot product)을 계산해 현재 단어가 다른 단어들과 얼마나 관련 있는지 점수로 만든다. 그 다음 이 점수를 $\sqrt{d_k}$ 로 나눈 뒤 softmax를 적용해 가중치로 바꾸고, 그 가중치로 Value를 가중합해 최종 출력을 만든다. 식으로 쓰면 다음과 같다.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

이 과정을 직관적으로 보면, Q와 K의 유사도는 "무엇을 얼마나 참고할지"를 정하는 점수이고, V는 실제로 가져올 정보이다. 즉, 현재 단어는 다른 단어들과의 관련도를 먼저 계산한 뒤, 관련도가 높은 단어의 정보를 더 많이 반영해 새로운 표현을 만든다.

$\sqrt{d_k}$ 로 나누는 이유는, Query와 Key의 차원이 커질수록 내적값이 함께 커질 수 있기 때문이다. 내적값이 너무 커지면 softmax 결과가 특정 값에 과도하게 치우치고, 그로 인해 gradient가 매우 작아져 학습이 불안정해질 수 있다. 이를 막기 위해 내적값을 $\sqrt{d_k}$ 로 나누어 값의 크기를 조정하고, attention 계산이 더 안정적으로 이루어지도록 한다.

Self-Attention

Self-Attention은 하나의 시퀀스 안에서 각 단어가 같은 시퀀스의 다른 모든 단어를 참고해 자신의 표현을 업데이트하는 방식이다. 이때 Query, Key, Value는 모두 동일한 입력으로부터 생성되며, 각 단어는 다른 단어들과의 관련도를 계산해 중요한 정보일수록 더 크게 반영한다.

이 과정은 Query, Key, Value 구조로 표현할 수 있다.

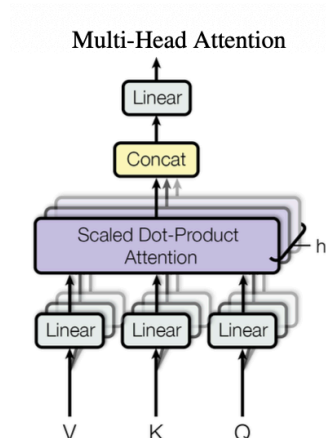
- Query (Q) : 현재 정보를 찾고 있는 기준이 되는 벡터
- Key (K) : 각 단어가 가진 특징을 나타내는 벡터
- Value (V) : 실제로 전달되는 정보

각 단어는 Query, Key, Value 벡터로 변환되고, 자신의 Query를 기준으로 다른 모든 단어의 Key와의 유사도를 계산한다. 이 유사도는 attention weight로 변환되며, 해당 가중치를 이용해 Value들을 가중합함으로써 새로운 표현을 만든다. 이를 통해 각 단어는 문장 전체의 문맥을 반영한 표현을 갖게 된다.

Encoder에서는 이러한 self-attention이 그대로 적용되어, 각 위치가 입력 문장의 모든 위치를 자유롭게 참고할 수 있다. 반면 Decoder에서는 미래 단어를 미리 참조하지 않도록 masking이 적용된 self-attention이 사용된다. 이를 통해 현재 시점의 출력이 이전 단어들에만 의존하도록 하여, 순차적인 생성 구조를 유지한다.

Self-Attention의 가장 큰 특징은 각 단어가 문장 전체를 한 번에 바라볼 수 있다는 점이다. 기존 RNN처럼 정보를 순차적으로 전달할 필요가 없기 때문에, 멀리 떨어진 단어 간의 관계도 직접적으로 반영할 수 있으며, 연산 또한 병렬적으로 수행할 수 있다. 이러한 특성 덕분에 Self-Attention은 Transformer의 핵심 구성 요소로 사용된다.

Multi-Head Attention



Multi-Head Attention은 하나의 attention을 여러 개로 확장해, 서로 다른 관점에서 정보를 동시에 바라보도록 만든 구조이다. 하나의 attention만 사용할 경우, 다양한 관계가 평균화되어 중요한 정보가 희석될 수 있기 때문에, 여러 개의 attention을 병렬로 수행해 더 풍부한 표현을 얻는다. 구체적으로는 입력으로 들어온 Query, Key, Value를 그대로 사용하는 것이 아니라, 각각을 여러 개의 서로 다른 선형 변환으로 투영해 여러 개의 Q_i , K_i , V_i 를 만든다. 그리고 각 쌍에 대해 Scaled Dot-Product Attention을 독립적으로 수행한다. 각 attention의 결과를 head라고 부르며, 여러 head의 결과를 이어 붙인(concatenate) 뒤 다시 한번 선형 변환을 적용해 최종 출력을 만든다. 이를 수식으로 표현하면 다음과 같다.

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

여기서 각 W_i^Q, W_i^K, W_i^V 는 서로 다른 projection 행렬이며, 각 head는 서로 다른 표현 공간에서 attention을 수행한다.

이 구조의 핵심은, 각 head가 서로 다른 관계를 학습할 수 있다는 점이다. 예를 들어 어떤 head는 문법적 관계를, 다른 head는 의미적 관계나 장거리 의존성을 더 잘 반영할 수 있다. 이런 방식으로 모델은 하나의 attention으로는 포착하기 어려운 다양한 패턴을 동시에 학습할 수 있다.

또한 각 head는 더 작은 차원에서 계산되기 때문에, 여러 개의 head를 사용하더라도 전체 계산량은 크게 증가하지 않는다. 결과적으로 Multi-Head Attention은 표현력을 높이면서도 계산 효율을 유지하는 구조라고 볼 수 있다.

Positional Encoding

Transformer는 RNN이나 CNN과 달리 단어의 순서를 직접 처리하는 구조가 없기 때문에, 문장 내 위치 정보를 별도로 주입해야 한다. 이를 위해 입력 embedding에 positional encoding을 더해, 각 단어가 문장에서 어떤 위치에 있는지를 모델이 알 수 있도록 한다.

Positional Encoding은 embedding과 동일한 차원 d_{model} 을 가지며, 두 벡터를 더하는 방식으로 결합된다. 이렇게 하면 단어의 의미 정보와 위치 정보가 동시에 반영된 표현이 encoder와 decoder의 입력으로 사용된다.

논문에서는 위치 정보를 표현하기 위해 사인(sin)과 코사인(cos) 함수를 사용한다. 각 위치 pos 와 차원 i 에 대해 다음과 같이 정의된다.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

이 방식은 각 차원이 서로 다른 주파수를 가지도록 하여, 위치 정보가 다양한 패턴으로 표현되도록 한다. 또한 이러한 구조를 사용하면 상대적인 위치 관계를 쉽게 학습할 수 있고, 학습 시보다 더 긴 문장에도 일반화할 수 있는 장점이 있다.

추가로, 학습 가능한 positional embedding을 사용하는 방법도 있지만, 실험 결과 두 방식은 비슷한 성능을 보였으며, 이 논문에서는 일반화 성능을 고려해 사인/코사인 기반 방식을 사용했다.

Why Self-Attention

Transformer에서 self-attention이 핵심 구조로 선택된 이유는, 기존의 RNN이나 CNN 기반 모델이 가지는 한계를 효과적으로 해결할 수 있기 때문이다. 이를 이해하기 위해서는 세 가지 관점에서 비교해볼 수 있다.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

1. 연산의 병렬화 가능성

RNN은 이전 시점의 계산 결과를 다음 시점에 전달하는 구조이기 때문에, 순차적으로 계산할 수밖에 없다. 즉, 시퀀스 길이에 비례해 $O(n)$ 의 sequential 연산이 필요하다. 반면 self-attention은 모든 단어 간의 관계를 한번에 계산할 수 있어, 대부분의 연산을 병렬적으로 수행할 수 있다. 이 덕분에 GPU를 활용한 학습에서 훨씬 높은 효율을 낼 수 있다.

2. 장거리 의존성(long-range dependency)을 학습하는 능력

자연어에서는 멀리 떨어진 단어 간의 관계가 중요한 경우가 많다. RNN은 이러한 정보를 여러 단계에 걸쳐 전달해야 하기 때문에, 정보가 점점 희석되거나 학습이 어려워질 수 있다. CNN 역시 한 번의 연산으로는 가까운 단어만 볼 수 있기 때문에, 멀리 떨어진 단어를 연결하려면 여러 층을 거쳐야 한다. 반면 self-attention은 모든 단어가 서로를 직접 참고할 수 있기 때문에, 두 단어 사이의 거리에 관계없이 한 번의 연산으로 관계를 반영할 수 있다. 즉, 정보 전달 경로(path length)가 매우 짧다는 장점이 있다.

3. 계산 복잡도 측면의 효율성

self-attention은 시퀀스 길이에 대해 $O(n^2)$ 의 연산을 필요로 하지만, 문장 길이 n 이 표현 차원 d 보다 작은 경우가 일반적이기 때문에 실제로는 RNN보다 더 빠르게 동작할 수 있다. 또한 행렬 연산으로 구현되기 때문에 하드웨어 최적화 측면에서도 유리하다. 매우 긴 시퀀스에서는 비용이 증가할 수 있지만, 이 경우 attention을 일부 구간으로 제한하는 방식으로 확장할 수 있다.

4. 해석 가능성(interpretability) 측면에서의 장점

attention weight를 통해 모델이 어떤 단어에 주목하고 있는지를 직접 확인할 수 있으며, 실제로 서로 다른 attention head가 문법적 관계나 의미적 구조를 학습하는 모습을 관찰할 수 있다.

이러한 이유들로 인해 self-attention은 기존의 순차적 또는 지역적 처리 방식 대신, 전체 문맥을 한 번에 고려하면서도 효율적으로 학습할 수 있는 구조로 자리 잡았으며, Transformer의 핵심 구성 요소가 되었다.

Conclusion

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Transformer는 기존 RNN 및 CNN 기반 모델보다 더 높은 성능을 달성하면서도, 더 적은 연산 비용으로 학습할 수 있음을 보여주었다. 실제로 기계 번역 작업에서 기존 모델들을 뛰어넘는 BLEU score를 기록했으며, 특히 Transformer (big) 모델은 최고 성능을 달성했다.

또한 주목할 점은, 이러한 성능 향상이 단순히 모델 크기 증가에 의한 것이 아니라, self-attention 기반 구조를 통해 더 효율적으로 문맥 정보를 활용한 결과라는 점이다. 기존 모델들이 더 많은 연산 비용을 필요로 했던 것과 달리, Transformer는 상대적으로 적은 계산량으로도 높은 성능을 달성했다.

이러한 결과는 Transformer가 단순한 성능 개선을 넘어, sequence modeling의 방식을 attention 중심으로 전환한 구조적 혁신임을 보여준다. 이후 BERT, GPT와 같은 모델들이 이 구조를 기반으로 발전하면서, Transformer는 자연어 처리의 핵심 아키텍처로 자리 잡게 되었다.



rachelses

얼레벌레 컴공댕이의 기록

0개의 댓글

06. 임베딩 — 순환 신경망(Recurrent Neural Network, RNN)

- RNN: 순서가 있는 연속적인 데이터(Sequence data)를 처리하기 위해 개발된 인공 신경망의 한 종류
 - 시퀀스 데이터: 길이 가변적, 순서에 따라 의미 有
 - 각 시점(Time step)의 데이터가 이전 시점의 데이터와 독립적이지 않음
→ 특정 시점 t 에서의 데이터가 이전 시점($t_0, t_1, \dots, t_{n-1}$)의 영향을 받음(연속형 데이터)
 - 이전에 처리한 데이터 → 다음 단계에 활용
 - 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태 예측하는 데 사용
- 시계열 데이터, 자연어 처리, 음성인식 및 기타 시퀀스 데이터와 같은 도메인에서 널리 사용
 - 자연어 데이터는 시퀀스 데이터의 일종
 - 각 단어는 이전에 등장한 단어의 영향을 받아 해당 문장의 의미 형성
 - 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 상관관계(Correlation) 존재

순환 신경망(RNN)

RNN은 시퀀스 데이터를 순서대로 입력받아 처리하며 각 시점마다 은닉상태(Hidden state)의 형태로 저장
RNN의 셀(Cell): 각 시점의 데이터를 입력으로 받아 hidden state와 출력값을 계산하는 노드

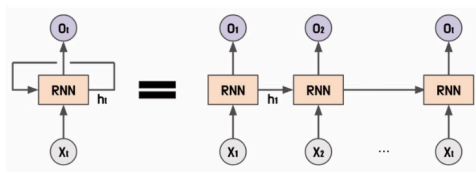


그림 6.12 순환 신경망의 셀

- 셀이 이전 시점의 은닉 상태 h_{t-1} 을 입력으로 받아 현재 시점의 은닉 상태 h_t 를 계산함

RNN의 Hidden state:

$$h_t = \sigma_h(h_{t-1}, x_t)$$
$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- σ_h : Activation func. 가중치(W)와 편향(b) 이용해 계산
- W_{hh} : h_{t-1} 에 대한 가중치
- W_{xh} : 입력값 x_t 에 대한 가중치
- b_h : h_t 의 편향

RNN의 output:

$$y_t = \sigma_y(h_t)$$
$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

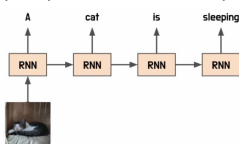
- σ_y : Activation func. h_t 를 입력으로 받아 y_t 계산
- W_{hy} : h_t 에 대한 가중치
- b_y : y_t 의 편향

RNN의 구조

일대다 구조(One-to-Many)

하나의 입력 시퀀스 → 여러 개의 출력값 생성

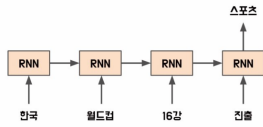
- 일대다 구조를 구현하려면 출력 시퀀스의 길이를 미리 알고 있어야 함
→ 입력 시퀀스를 처리하면서 시퀀스의 정보를 활용해 출력 시퀀스의 길이를 예측하는 모델을 같이 구현해야 함
- e.g.1, 자연어 처리 분야
 - (입력) 문장 → (출력) 각 단어의 품사
 - 문장에서 각 단어의 품사 예측하는 작업
- e.g.2, 이미지 캡셔닝(Image Captioning) 모델
 - (입력) 이미지 데이터 → (출력) 이미지에 대한 설명



다대일 구조(Many-to-One)

여러 개의 입력 시퀀스 → 하나의 출력값 생성

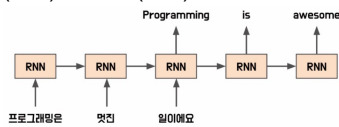
- e.g.1, 감성 분류(Sentiment Analysis)
 - 특정 문장의 감정(긍정, 부정)을 예측하는 작업
 - (입력) 문장 → (출력) 해당 문장의 감정(긍정/부정)
- e.g.2, 자연어 추론
 - 입력 시퀀스가 어떤 범주에 속하는지 구분하는 문장 분류



다대다 구조(Many-to-Many)

여러 개의 입력 시퀀스 → 여러 개의 출력 시퀀스

- 입력 시퀀스와 출력 시퀀스의 길이가 서로 다를 수 있음(입력 문장의 길이 ≠ 출력 문장의 길이)
→ 서로의 길이를 맞추기 위해 패딩을 추가하거나 잘라내는 등의 전처리 과정 필요
- Seq2Seq 구조로 이루어짐: Encoder(입력 시퀀스 처리) + Decoder(출력 시퀀스 생성)
 - (인코더) 입력 시퀀스 처리 → (출력) 고정 크기의 벡터 (입력) → (디코더) 출력 시퀀스 생성
- e.g.1, 번역기
 - (입력) 문장 → (출력) 번역된 문장

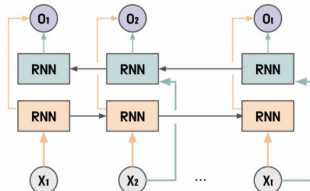


- e.g.2, 음성 인식기
 - (입력) 음성 신호 → (출력) 문장

양방향 순환 신경망(Bidirectional Recurrent Neural Network, BiRNN)

기본 RNN의 시간 방향을 양방향으로 처리 가능

→ 이전 시점($t-1$) + 이후 시점($t+1$)의 hidden state 이용



- 입력 데이터를 순방향으로 처리하는 것만 아니라 역방향으로 거꾸로 읽어들이어 처리하는 방식
- 대부분의 시퀀스 데이터는 이전/이후 시점 데이터와 큰 상관관계를 갖고 있기 때문에 BiRNN(양방향적 정보) 이용
- e.g., "인생은 B와 _ 사이의 C다."
 - 위 문장에서 _에 입력될 단어를 예측해야 한다면 앞/뒤 문장 모두의 영향을 받음

다중 순환 신경망(Stacked Recurrent Neural Network)

여러 개의 RNN을 연결해 구성한 모델(여러 개의 RNN 층으로 구성). 각 RNN이 서로 다른 정보를 처리

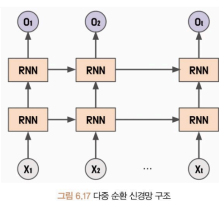


그림 6.17 다중 순환 신경망 구조

- 각 층에서의 출력값 → 다음 층으로 전달
- 다양한 특징을 추출할 수 있어 성능 향상 O
- 층 깊어질수록 더 복잡한 패턴 학습 가능 BUT 층 많아질수록 학습 시간 오래걸리고, gradient vanishing 문제 발생 가능성 높아짐

RNN 클래스

```
# 순환 신경망 클래스
```

```
rnn = torch.nn.RNN(
```

```

    input_size,          # 필수
    hidden_size,         # 필수
    num_layers=1,        # RNN 층수(2 이상이면 다중RNN)
    nonlinearity="tanh",  # Activation Func.("tanh"/"relu")
    bias=False,          # b_h 항 사용여부
    batch_first=True,
    dropout=0,
    bidirectional=False
)

```

양방향 다층 신경망

```

import torch
from torch import nn

input_size = 128
output_size = 256
num_layers = 3
bidirectional = True

model = nn.RNN(
    input_size=input_size,
    hidden_size=output_size,
    num_layers=num_layers,
    nonlinearity="tanh",
    batch_first=True,
    bidirectional=bidirectional
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, output_size) # 초기 은닉 상태: [계층 수 * 양방향 여부 + 1, 배치 크기, 은닉 상태 크기]

outputs, hidden = model(inputs, h_0) # forward propagation
print(outputs.shape)
print(hidden.shape)

''' 출력 결과
torch.Size([4,6,512])
torch.Size([6,4,256])
'''

```

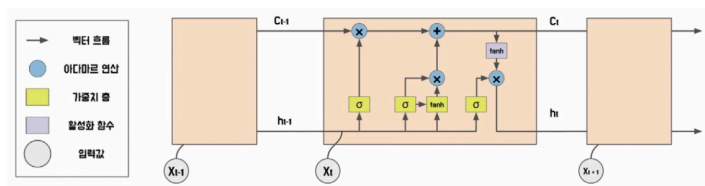
장단기 메모리(Long Short-Term Memory, LSTM)

기존 RNN이 갖고 있던 기억력 부족과 gradient vanishing 문제 해결한 모델

- 장기 의존성 문제(Long-term dependencies): 특정 시점에서 이전 입력 데이터의 정보를 이용해 출력값 예측 → 시간적으로 먼 과거의 정보는 잘 기억 X
- 활성화 함수 문제: tanh, ReLU 함수의 특성으로 인해 backpropagation 과정에서 기울기 소실/폭주 발생 가능성 존재
⇒ 이를 해결하기 위해 장단기 메모리(RNN + Memory cell + Gate) 사용

LSTM 구조

핵심 구조

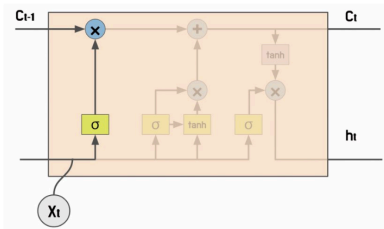


- 셀 상태(Cell State): 정보를 저장, 유지하는 메모리 역할. 출력 게이트와 망각 게이트에 의해 제어됨
- 3개의 게이트
 - 망각 게이트(Forget gate): 이전 셀 상태에서 어떤 정보를 삭제할지 결정

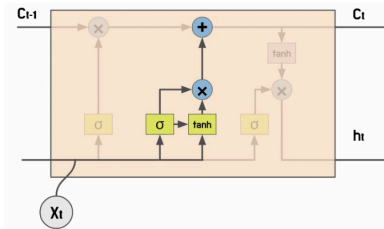
- 기억 게이트(Input gate): 새로운 정보를 어떤 부분에 추가할지 결정. 현재 입력과 이전 셀 상태를 입력으로 받음(→ 어떤 정보 추가할지 결정)
- 출력 게이트(Output gate): 셀 상태의 정보 중 어떤 부분을 출력할지 결정. 현재 입력 + 이전 셀 상태 + 새로 추가된 정보를 입력으로 받아 출력할 정보 결정

전체 흐름

1. 망각 게이트(Forget Gate)



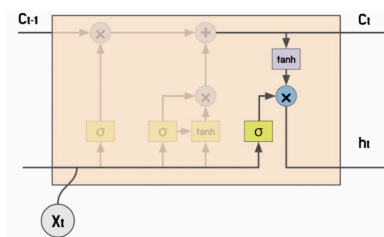
2. 기억 게이트(Input Gate)



3. 메모리 셀 업데이트

- 망각 게이트 + 기억 게이트 결과를 합산해 현재 메모리 셀 계산
- $c_t = f_t \odot c_{t-1} + g_t \odot i_t$
 - $f_t \odot c_{t-1}$: 이전 기억 중 유지할 부분
 - $g_t \odot i_t$: 새로 추가할 부분
 - $\odot$: 아다마르 곱 (원소별 곱셈)

4. 출력 게이트



5. 은닉 상태 갱신

- $h_t = o_t \odot \tanh(c_t)$
 - o_t : 출력 게이트 (0~1)
 - $\tanh(c_t)$: 메모리 셀을 -1 ~ 1로 정규화
 - 두 값의 아다마르 곱 → 이전 은닉 상태의 영향도 제어

LSTM 클래스

```
# 장단기 메모리 클래스
# LSTM은 활성화함수(nonlinearity) 사용 X
lstm = torch.nn.LSTM(
    input_size,
    hidden_size,
    num_layers=1,
    bias=False,
    batch_first=True,
    dropout=0,
    bidirectional=False,
    proj_size=0 # 투사 크기. LSTM 메모리 계층의 출력에 대한 선형 투사(Linear Projection) 크기 결정. hidden_size보다 작게 설정
)
```

```
# 양방향 다층 장단기 메모리
import torch
from torch import nn
```

```

input_size = 128
ouput_size = 256
num_layers = 3
bidirectional = True
proj_size = 64

model = nn.LSTM(
    input_size=input_size,
    hidden_size=ouput_size,
    num_layers=num_layers,
    batch_first=True,
    bidirectional=bidirectional,
    proj_size=proj_size,
)

batch_size = 4
sequence_len = 6

inputs = torch.randn(batch_size, sequence_len, input_size)
h_0 = torch.rand(
    num_layers * (int(bidirectional) + 1),
    batch_size,
    proj_size if proj_size > 0 else ouput_size,
)
c_0 = torch.rand(num_layers * (int(bidirectional) + 1), batch_size, ouput_size)

outputs, (h_n, c_n) = model(inputs, (h_0, c_0))

print(outputs.shape)
print(h_n.shape)
print(c_n.shape)

```

07. 트랜스포머 — Transformer

- 딥러닝 모델의 하나로, 기계 번역·챗봇·음성 인식 등 다양한 자연어 처리 분야에서 성과를 내는 모델
- 순환 신경망(RNN)이나 합성곱 신경망(CNN)과 달리 어텐션 메커니즘만 사용해 시퀀스 임베딩을 표현
- 이러한 장점으로 자연어 처리 분야에서 가장 인기 있는 모델

어텐션 메커니즘의 역할

- 인코더와 디코더 간의 상호작용으로 입력 시퀀스의 중요한 부분에 초점을 맞춰 문맥을 이해하고 적절한 출력을 생성
- 인코더: 입력 시퀀스를 임베딩하여 고차원 벡터로 변환
- 디코더: 인코더의 출력을 입력으로 받아 출력 시퀀스를 생성
- 어텐션은 인코더·디코더 단어 사이의 상관관계를 계산해 중요한 정보에 집중
- 결과적으로 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 단어와 관련 있는지 파악 → 번역, 요약문 생성 등의 작업 수행 가능

기존 모델 대비 장점

- 순환 신경망 기반 모델보다 학습 속도가 빠름
- 병렬 처리 가능 → 대규모 데이터셋에서 높은 성능
- 임베딩 과정에서 문장의 전체 정보를 고려하므로 문장이 길어져도 성능이 유지됨

전체 구조

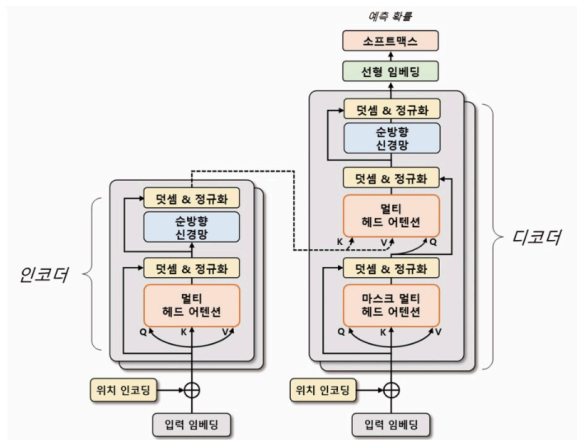


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

- 인코더와 디코더 두 부분으로 구성, 각각 **N**개의 트랜스포머 블록으로 구성
- 트랜스포머 블록 = 멀티 헤드 어텐션 + 순방향 신경망
- 그림 7.2 기준 데이터 흐름
 - 공통 입력: 입력 임베딩 + 위치 인코딩
 - 인코더 블록: 멀티 헤드 어텐션(Q, K, V 모두 자기 입력) → 덧셈 & 정규화 → 순방향 신경망 → 덧셈 & 정규화
 - 디코더 블록: 마스크 멀티 헤드 어텐션 → 덧셈 & 정규화 → 멀티 헤드 어텐션(K, V는 인코더 출력, Q는 디코더) → 덧셈 & 정규화 → 순방향 신경망 → 덧셈 & 정규화
 - 출력단: 선형 임베딩 → 소프트맥스 → 예측 확률

멀티 헤드 어텐션

- 입력 시퀀스에서 쿼리(Q), 키(K), 값(V) 벡터를 정의해 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
- 계산 과정
 - 쿼리와 각 키의 유사도를 계산
 - 해당 유사도를 가중치로 사용해 값 벡터를 합산
- 이렇게 계산된 어텐션 행렬이 입력 시퀀스 각 단어의 임베딩 벡터를 대체
- 즉, 단어 사이의 상호작용을 고려해 임베딩 벡터를 갱신

순환 신경망

- 어텐션에서 산출된 임베딩 벡터를 더욱 고도화하기 위해 사용
- 여러 개의 선형 계층으로 구성: 입력 벡터에 가중치를 곱하고, 편향을 더하고, 활성화 함수를 적용
- 학습된 가중치들은 입력 시퀀스 각 단어의 의미를 잘 파악할 수 있는 방식으로 갱신됨

소스 / 타겟 데이터

- 입력 시퀀스 데이터를 소스(Source) 와 타겟(Target) 으로 나눠 처리
- 예: 영어 → 한글 번역
 - 타겟 = 생성하는 언어인 한글
 - 소스 = 참조하는 언어인 영어

인코더, 디코더 동작

- 인코더
 - 소스 시퀀스 데이터를 위치 인코딩(Positional Encoding)된 입력 임베딩으로 표현
 - 트랜스포머 블록의 출력 벡터를 생성하며, 이 벡터는 입력 시퀀스 데이터의 관계를 잘 표현하도록 구성
- 디코더
 - 인코더와 유사하게 트랜스포머 블록으로 구성
 - 단, 마스크 멀티 헤드 어텐션(Masked Multi-Head Attention) 을 사용해 타겟 시퀀스를 순차적으로 생성
 - 디코더 입력 시퀀스들의 관계를 고도화하기 위해 인코더의 출력 벡터 정보를 참조
- 최종 출력
 - 디코더 출력 벡터는 선형 임베딩으로 재표현되어 이미지나 자연어 모델에 활용
 - 이 과정을 통해 챗봇, 기계 번역, 음성 인식 등에서 좋은 성능을 보임

입력 임베딩과 위치 인코딩

입력 임베딩, 위치 정보의 필요성

- 트랜스포머에서 입력 시퀀스의 각 단어는 임베딩 처리되어 벡터 형태로 변환됨
- 순환 신경망과 달리 입력 시퀀스를 병렬 구조로 처리하기 때문에 단어의 순서 정보가 제공되지 않음
- 따라서 위치 정보를 임베딩 벡터에 추가해 순서 정보를 모델에 반영해야 함 → 이를 위해 위치 인코딩(Positional Encoding) 사용

위치 인코딩

- 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
- 각 단어의 위치 정보를 나타내는 벡터를 더하여 임베딩 벡터에 위치 정보를 반영
- 위치 인코딩 벡터는 **sin 함수와 cos 함수**로 생성되며, 이를 통해 임베딩 벡터 + 위치 정보가 결합된 최종 입력 벡터가 만들어짐
- 각 토큰의 위치를 **각도로 표현**해 sin/cos 함수로 벡터를 계산
- 토큰의 위치마다 동일한 임베딩 벡터를 사용하지 않기 때문에, 모델이 각 토큰의 위치 정보를 학습할 수 있음
- 트랜스포머에서 입력 시퀀스의 순서 정보를 보존하기 위한 중요한 방법 중 하나

수식

$$PE(pos, 2i) = \sin\left(pos/10000^{2i/d_{model}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(pos/10000^{2i/d_{model}}\right)$$

- **pos**: 입력 시퀀스에서 해당 단어의 위치
- **i**: 임베딩 벡터의 차원 인덱스
- 차원 인덱스가 짝수면 sin 수식, 홀수면 cos 수식을 적용
- $1/10000^{2i/d_{model}}$: 각도 정보로 변환하기 위한 스케일링 인자, 각 위치에 대한 주기적인 신호를 생성

특수 토큰

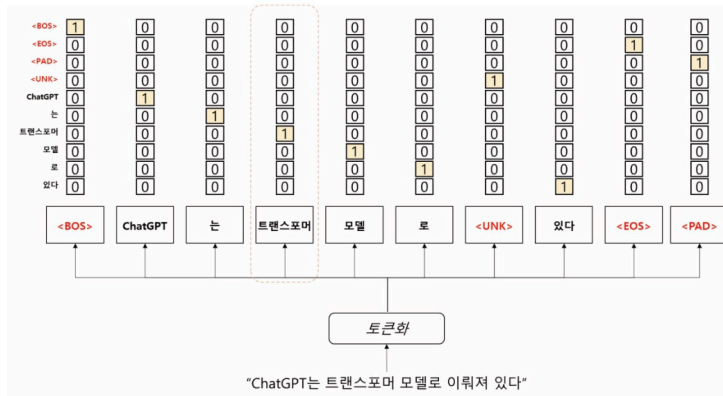
- 트랜스포머는 단어 토큰 이외의 특수 토큰을 활용해 문장을 표현
- 용도
 - 입력 시퀀스의 시작과 끝을 나타냄
 - 마스킹(Masking) 영역으로 사용
- 효과
 - 모델이 입력 시퀀스의 시작과 끝을 인식할 수 있게 함
 - 마스킹을 통해 일부 입력을 무시할 수 있음
 - 예: 번역 모델에서 디코더 입력 시퀀스의 현재 위치 이후 토큰을 마스킹해 이전 토큰만 참조

토큰	의미	역할
BOS	Beginning of Sentence	문장의 시작을 나타냄

토큰	의미	역할
EOS	End of Sentence	문장의 종료를 나타냄
UNK	Unknown	어휘 사전에 없는 모르는 단어. 모델이 이전에 본 적 없는 단어를 처리할 때 사용
PAD	Padding	모든 문장을 일정한 길이로 맞추기 위해 사용. 짧은 문장의 빈 공간을 채움

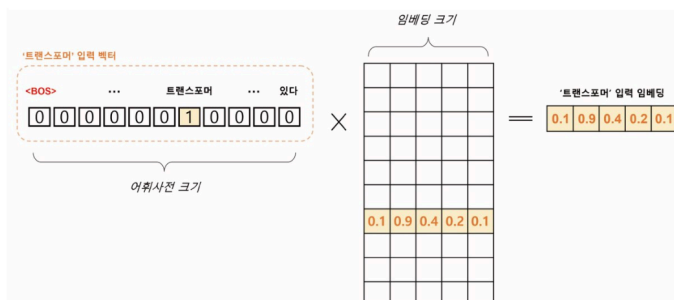
- BOS/EOS는 문장의 시작과 끝을 **명확히** 하기 위해 사용
- 모두 자연어 처리에서 일반적으로 사용되는 특수 토큰

문장 토큰 배열



- 예시 문장: "ChatGPT는 트랜스포머 모델로 이뤄져 있다"
- 토큰화 결과 배열: <BOS> → ChatGPT → 는 → 트랜스포머 → 모델 → 로 → <UNK> → 있다 → <EOS> → <PAD>
- 각 토큰은 어휘 사전에 등장하는 위치에 **원-핫 인코딩**으로 표현됨 (해당 위치만 1, 나머지는 0)
- 사전에 없는 단어("이뤄져")는 <UNK> 로 대체

입력 임베딩 산출 과정



- 원-핫 벡터 × 임베딩 행렬 = 입력 임베딩 벡터
- 변환 과정은 **Word2Vec** 방법과 동일
- 차원 정리 (어휘 사전 크기 V, 입력 임베딩 차원 d)
 - '트랜스포머'의 원-핫 벡터: **[1, V]**
 - 임베딩 행렬: **[V, d]**
 - 결과 벡터: **[1, d]**
- 일반화: N개의 문장이 최대 S개의 토큰 길이를 가질 때
 - 원-핫 벡터 텐서 **[N, S, V]** → 임베딩 텐서 **[N, S, d]**
- 이 임베딩 텐서는 입력으로 사용되며, 트랜스포머의 모든 계층에서 공유되는 텐서로 사용됨